CNN Parameter Counting – Section 1: Mastering Parameter Calculation Across 10 Architectures

Introduction

Every CNN is built from layers, and every layer has a fixed number of learnable numbers called **parameters** (weights and biases). Knowing exactly how many parameters a model has tells you how much memory it needs, how long it will take to train, and whether it will overfit on small datasets. This section teaches you to count parameters precisely across ten progressively complex CNN architectures – from the simplest possible network to depthwise-separable convolutions and multi-branch designs.

Architectures covered:

#	Architecture	Key Scenario
1	Conv → Flatten → Dense	Single conv, fully-connected head
2	Conv → Pool → Flatten → Dense	Adding max pooling
3	Conv → Conv → Flatten → Dense	Stacked convolutions
4	Conv → Pool → Conv → Pool → Flatten → Dense	Classic LeNet-style
5	Conv → Pool → Flatten → Dense → Dense	Two FC layers at end
6	Conv → BN → Pool → Flatten → Dense	Batch normalisation
7	Conv → GAP → Dense	Global Average Pooling (no flatten)
8	Conv → Conv → GAP → Dense	Deep conv + GAP
9	Depthwise-Sep Conv → GAP → Dense	Depthwise-separable convolution
10	Two-branch → Concat → GAP → Dense	Multi-branch (Inception-style)

Core Formula Reference

Before diving into architectures, master these four formulas. Every calculation in this section uses them.

What a Parameter Is

A parameter is a number the network learns during training. In a convolutional layer, each filter is a small grid of weights – one per kernel position per input channel – plus one bias value per filter. In a dense layer, every output neuron connects to every input value, plus one bias per neuron.

Formula 1 – Convolutional layer:

$$\text{Params}_{\text{conv}} = (K \times K \times C_{\text{in}} + 1) \times C_{\text{out}}$$

- K: kernel (filter) size in one dimension (e.g., 3 for a 3×3 kernel)
- $K \times K$: total weights per input channel per filter
- C_{in}: number of input channels (depth of the input feature map)
- $+1$: one bias per output filter
- C_{out}: number of filters (= number of output channels)

Formula 2 – Dense (fully connected) layer:

$$\text{Params}_{\text{dense}} = (N_{\text{in}} + 1) \times N_{\text{out}}$$

- N_{in}: number of input values (flattened size or previous layer's units)
- $+1$: one bias per output neuron
- N_{out}: number of neurons in this layer

Formula 3 – Batch Normalisation layer:

$$\text{Params}_{\text{BN}} = 4 \times C$$

- C: number of channels (or features) being normalised
- The 4 parameters per channel are: γ (scale), β (shift), μ (running mean), σ^2 (running variance)
- Note: μ and σ^2 are not trained by gradient descent but are counted as stored parameters

Formula 4 – Depthwise Separable Convolution (two sub-layers):

$$\text{Params}_{\text{DW}} = (K \times K + 1) \times C_{\text{in}}$$

$$\text{Params}_{\text{PW}} = (1 \times 1 \times C_{\text{in}} + 1) \times C_{\text{out}}$$

- Depthwise (DW): one $K \times K$ filter per input channel, so C_{in} filters total
- Pointwise (PW): a 1×1 conv that mixes channels

Zero-parameter layers (no learnable parameters):

- Max Pooling, Average Pooling, Global Average Pooling, Flatten, Dropout, ReLU

Output spatial size after a Conv or Pool (no padding, stride 1):

$$H_{\text{out}} = H_{\text{in}} - K + 1 \quad \text{(valid padding)}$$

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} - K}{S} \right\rfloor + 1 \quad \text{(general, stride } S\text{)}$$

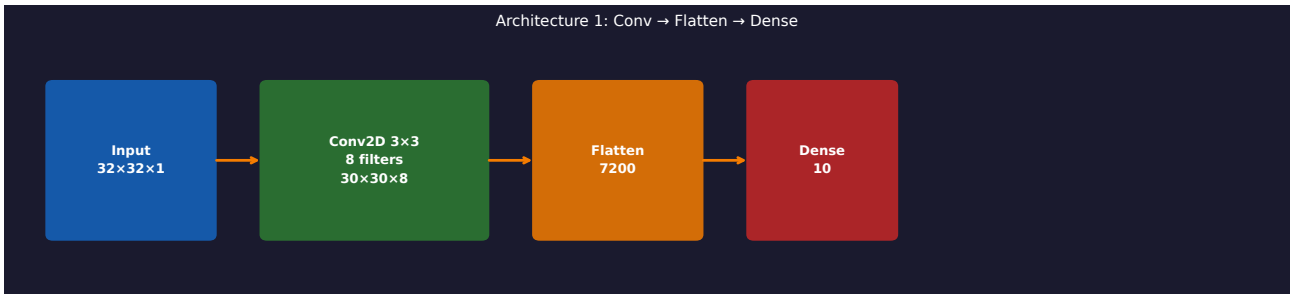
Architecture 1 – Conv → Flatten → Dense

Concept

The most fundamental CNN: one convolutional layer extracts features, Flatten converts the 3D feature map into a 1D vector, and a Dense layer produces class scores. There is no pooling – the spatial information flows directly into the fully-connected head.

Real-world analogy: Think of the conv layer as a team of inspectors, each carrying a different 3×3 magnifying glass (filter), sliding it over every position of the image. Each inspector produces one number per position. Flatten lines all their notes into one long list. The Dense layer then reads the entire list and votes on the class.

Visualization



Mathematical Foundation

Intuition: Each of the 8 filters in the Conv layer has a 3×3 grid of weights (9 numbers) and must look at one input channel (1 channel), so each filter has $9 \times 1 = 9$ weights, plus 1 bias = 10 numbers. Eight filters → 80 numbers. After convolution, the spatial size shrinks from 32×32 to 30×30 (because a 3×3 kernel with no padding reduces each dimension by 2). Flattening gives $30 \times 30 \times 8 = 7200$ inputs to the Dense layer. Each of the 10 Dense neurons connects to all 7200 inputs plus has 1 bias.

Conv layer:

$$\text{Params}_{\text{conv}} = (3 \times 3 \times 1 + 1) \times 8 = 10 \times 8 = 80$$

Dense layer:

$$\text{Params}_{\text{dense}} = (7200 + 1) \times 10 = 72{,}010$$

Total:

$$80 + 72{,}010 = \boxed{72{,}090}$$

Numerical Example – Full Worked Steps

Step	Calculation	Result
Conv: weights per filter	$3 \times 3 \times 1 = 9$	9
Conv: +bias per filter	$9 + 1 = 10$	10
Conv: × 8 filters	10×8	**80**
Output spatial size	$32 - 3 + 1 = 30 \rightarrow 30 \times 30$	900 per channel
Flatten size	$30 \times 30 \times 8 = 7200$	7200
Dense: weights	$7200 \times 10 = 72{,}000$	72,000
Dense: +biases	$10 \times 1 = 10$	10
Total	$80 + 72{,}000 + 10$	**72,090**

Interpretation: 72,090 parameters – nearly all (99.9%) live in the Dense layer. This is why pooling exists: to shrink the spatial size before the FC head and avoid enormous parameter counts.

Code

```python
import tensorflow as tf
from tensorflow import keras

model = keras.Sequential([
    # Input: 32×32 grayscale image
    keras.layers.Input(shape=(32, 32, 1)),

    # Conv layer: 8 filters, 3×3 kernel, no padding → output 30×30×8
    # Params = (3×3×1 + 1) × 8 = 80
    keras.layers.Conv2D(filters=8, kernel_size=3, activation='relu'),

    # Flatten 30×30×8 → 7200 values
    keras.layers.Flatten(),

    # Dense output: 10 classes
    # Params = (7200 + 1) × 10 = 72,010
    keras.layers.Dense(10, activation='softmax'),
])

model.summary()
# Output:
# Layer (type)          Output Shape       Param #
# conv2d (Conv2D)       (None, 30, 30, 8)      80
# flatten (Flatten)     (None, 7200)            0
# dense (Dense)         (None, 10)          72,010
# Total params: 72,090
```

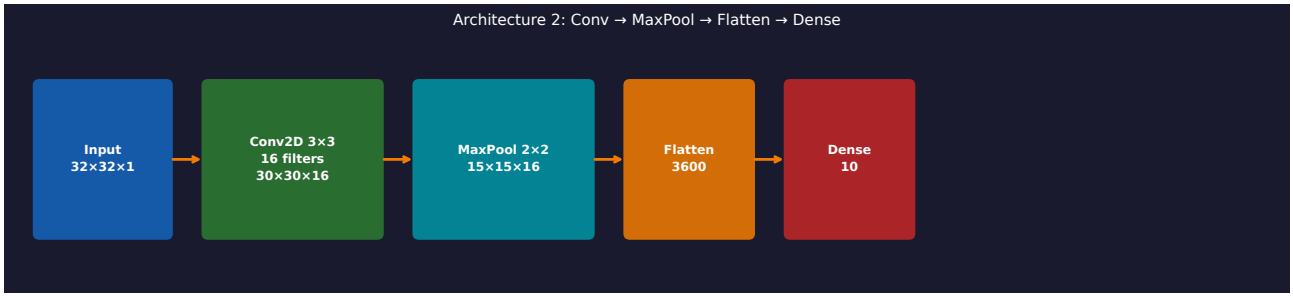
Architecture 2 – Conv → MaxPool → Flatten → Dense

Concept

Adding a MaxPool layer after the conv is the classic trick to reduce spatial dimensions before the FC head. Pooling has zero parameters – it simply picks the maximum value in each window.

Analogy: After the inspectors finish noting their observations (conv), a supervisor reviews each 2×2 patch of notes and keeps only the most notable finding (max pool). This cuts the report length by 4×, making the final reading step (Dense) far cheaper.

Visualization



Mathematical Foundation

Intuition: The conv layer now uses 16 filters. MaxPool 2×2 halves each spatial dimension (30→15), so the flatten size drops from 7200 to 3600. The Dense layer is now much cheaper.

Conv layer:

$$\text{Params}_{\text{conv}} = (3 \times 3 \times 1 + 1) \times 16 = 10 \times 16 = 160$$

MaxPool: 0 parameters.

Dense layer:

$$\text{Params}_{\text{dense}} = (3600 + 1) \times 10 = 36{,}010$$

Total:

$$160 + 0 + 36{,}010 = \boxed{36{,}170}$$

Numerical Example – Full Worked Steps

Step	Calculation	Result
Conv: weights per filter	$3 \times 3 \times 1 = 9$	9
Conv: +bias	$9 + 1 = 10$	10
Conv: × 16 filters	10×16	**160**
After conv spatial	$32 - 3 + 1 = 30$	30×30
After MaxPool 2×2	$30/2 = 15$	15×15
Flatten size	$15 \times 15 \times 16 = 3600$	3600
Dense weights	$3600 \times 10 = 36{,}000$	36,000
Dense biases	10	10
Total	$160 + 36{,}010$	**36,170**

Interpretation: By inserting one MaxPool 2×2, the Dense layer dropped from 72,010 → 36,010 params – halved at zero cost. Pooling is a free compression step.

Code

```
model = keras.Sequential([
    keras.layers.Input(shape=(32, 32, 1)),

    # Conv: (3×3×1 + 1) × 16 = 160 params
    keras.layers.Conv2D(16, kernel_size=3, activation='relu'),

    # MaxPool 2×2: 0 params, halves spatial dims 30→15
    keras.layers.MaxPooling2D(pool_size=2),

    # Flatten 15×15×16 = 3600
    keras.layers.Flatten(),

    # Dense: (3600 + 1) × 10 = 36,010 params
    keras.layers.Dense(10, activation='softmax'),
])
```

```
model.summary()
# Total params: 36,170
```

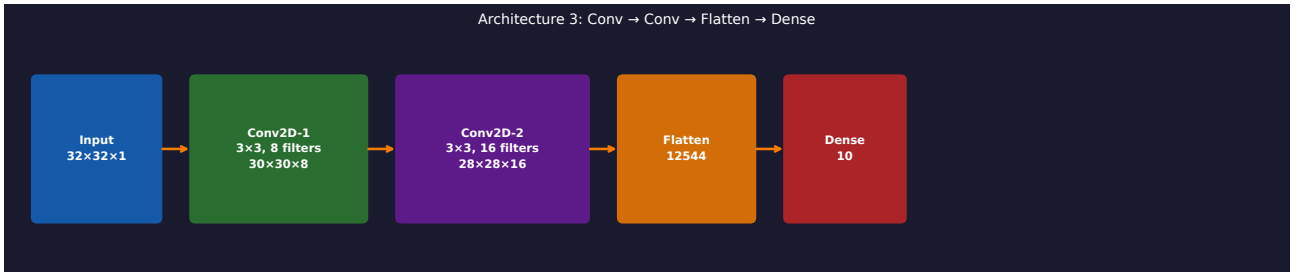
Architecture 3 – Conv → Conv → Flatten → Dense

Concept

Stacking two conv layers (without pooling) lets the second layer see combinations of features extracted by the first. The first conv detects edges; the second can detect corners or textures. The key rule: **the second conv's C_{in} equals the first conv's C_{out}.**

Analogy: The first team of inspectors finds raw features (edges). A second team of inspectors then examines the first team's reports – their input channels are the first team's output channels.

Visualization



Mathematical Foundation

Intuition: Conv-1 takes in 1 channel, produces 8. Conv-2 takes in those 8 channels – so each of its 16 filters must look at all 8 input channels, meaning $3 \times 3 \times 8 = 72$ weights per filter.

Conv-1:

$$\text{Params}_1 = (3 \times 3 \times 1 + 1) \times 8 = 10 \times 8 = 80$$

Conv-2 ($C_{\text{in}} = 8$):

$$\text{Params}_2 = (3 \times 3 \times 8 + 1) \times 16 = 73 \times 16 = 1{,}168$$

Flatten size: After Conv-1: $32-3+1 = 30$. After Conv-2: $30-3+1 = 28$. So $28 \times 28 \times 16 = 12{,}544$.

Dense:

$$\text{Params}_{\text{dense}} = (12544 + 1) \times 10 = 125{,}450$$

Total:

$$80 + 1{,}168 + 125{,}450 = \boxed{126{,}698}$$

Numerical Example – Full Worked Steps

Step	Calculation	Result
Conv-1 params	$(9 \times 1 + 1) \times 8$	**80**
Spatial after Conv-1	$32 - 3 + 1 = 30$	30×30
Conv-2 params	$(9 \times 8 + 1) \times 16 = 73 \times 16$	**1,168**
Spatial after Conv-2	$30 - 3 + 1 = 28$	28×28
Flatten size	$28 \times 28 \times 16$	12,544
Dense params	$(12544 + 1) \times 10$	**125,450**
Total	$80 + 1{,}168 + 125{,}450$	**126,698**

Interpretation: Without pooling, two convs leave a very large spatial map feeding the Dense layer. The conv layers themselves are cheap (1,248 params total); the Dense dominates again (99% of params).

Code

```
model = keras.Sequential([
    keras.layers.Input(shape=(32, 32, 1)),

    # Conv-1: (3×3×1 + 1) × 8 = 80 params
    keras.layers.Conv2D(8, kernel_size=3, activation='relu'),

    # Conv-2: C_in=8 → (3×3×8 + 1) × 16 = 1,168 params
    keras.layers.Conv2D(16, kernel_size=3, activation='relu'),

    # Flatten: 28×28×16 = 12,544
    keras.layers.Flatten(),

    # Dense: (12544 + 1) × 10 = 125,450
    keras.layers.Dense(10, activation='softmax'),
])

model.summary()
# Total params: 126,698
```

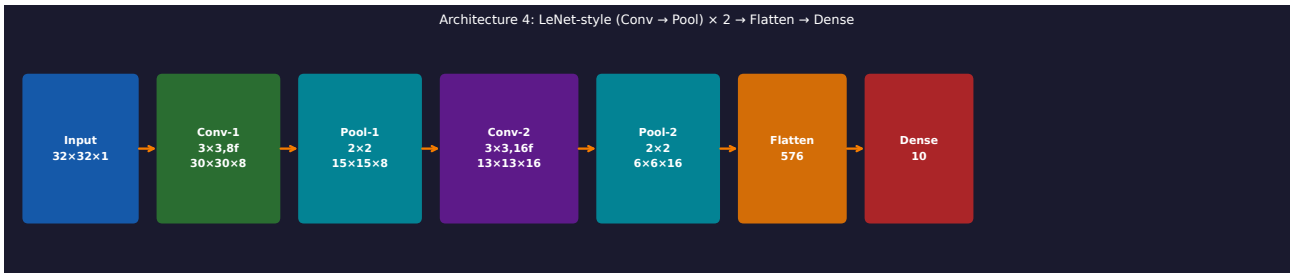
Architecture 4 – Conv → Pool → Conv → Pool → Flatten → Dense (LeNet-style)

Concept

This is the classic LeNet-5 pattern: alternate conv and pooling blocks, then a dense head. Each pooling halves the spatial size, so by the time we flatten, the map is much smaller. This is the template behind modern classification CNNs.

Analogy: First the inspector team scans the full image (Conv-1), then a supervisor summarises each 2×2 patch (Pool-1). A second team then analyses those summaries at a higher level (Conv-2), and a second supervisor compresses again (Pool-2). Only then does the final judge (Dense) make a decision.

Visualization



Mathematical Foundation

Intuition: Conv-1 input is 1 channel → 8 channels. MaxPool-1 halves 30→15. Conv-2 input is 8 channels → 16 channels. MaxPool-2 halves 13→6 (floor(13/2)=6). Flatten: 6×6×16=576. Dense final.

Conv-1:

$$\text{Params}_1 = (3 \times 3 \times 1 + 1) \times 8 = 80$$

Conv-2 ($C_{\text{in}} = 8$):

$$\text{Params}_2 = (3 \times 3 \times 8 + 1) \times 16 = 73 \times 16 = 1{,}168$$

Dense:

$$\text{Params}_{\text{dense}} = (576 + 1) \times 10 = 5{,}770$$

Total:

$$80 + 1{,}168 + 5{,}770 = \boxed{7{,}018}$$

Numerical Example – Full Worked Steps

Step	Calculation	Result
Conv-1 params	$(9 \times 1 + 1) \times 8$	**80**
After Conv-1	$32 - 3 + 1 = 30$	30×30×8
After Pool-1	$\lfloor 30/2 \rfloor = 15$	15×15×8
Conv-2 params	$(9 \times 8 + 1) \times 16$	**1,168**
After Conv-2	$15 - 3 + 1 = 13$	13×13×16
After Pool-2	$\lfloor 13/2 \rfloor = 6$	6×6×16
Flatten	$6 \times 6 \times 16 = 576$	576
Dense params	$(576 + 1) \times 10$	**5,770**
Total	$80 + 1{,}168 + 5{,}770$	**7,018**

Interpretation: Two pooling stages reduced the Dense layer from 125,450 (Architecture 3) to 5,770 – a **22× reduction** in the FC head. The total dropped from 126,698 to 7,018. Pooling is extremely powerful for controlling parameter count.

Code

```
model = keras.Sequential([
    keras.layers.Input(shape=(32, 32, 1)),

    keras.layers.Conv2D(8, kernel_size=3, activation='relu'),    # 80
    ↪ params
    keras.layers.MaxPooling2D(2),                                  # 0
    ↪ params

    keras.layers.Conv2D(16, kernel_size=3, activation='relu'),   # 1,168
    ↪ params
    keras.layers.MaxPooling2D(2),                                  # 0
    ↪ params

    keras.layers.Flatten(),                                        # 576
    ↪ units
    keras.layers.Dense(10, activation='softmax'),                  # 5,770
    ↪ params
])

model.summary()
# Total params: 7,018
```

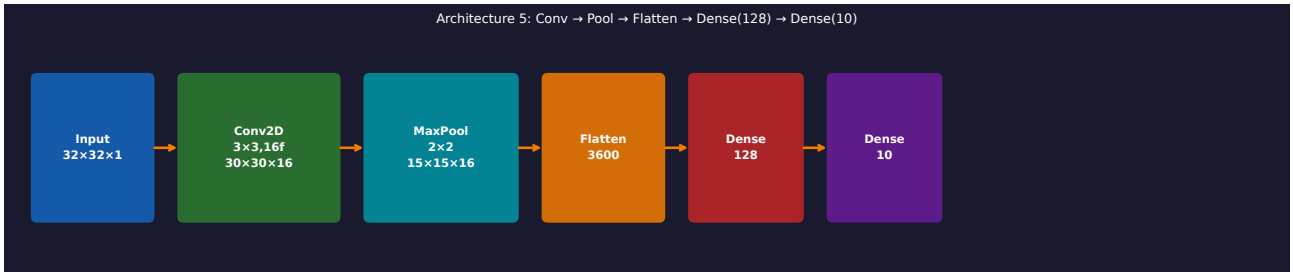
Architecture 5 – Conv → Pool → Flatten → Dense(128) → Dense(10)

Concept

Many real architectures place **two Dense layers** at the end: a large hidden Dense layer (e.g., 128 units) learns a rich intermediate representation, and a smaller output Dense layer produces class scores. The hidden Dense layer is often the single biggest parameter cost in the whole network.

Analogy: After the inspectors produce a compressed report (conv + pool + flatten), a first analyst (Dense-128) synthesises all 3600 inputs into 128 key conclusions. A second analyst (Dense-10) then reads those 128 conclusions and makes the final classification.

Visualization



Mathematical Foundation

Conv layer (same as Architecture 2):

$$\text{Params}_{\text{conv}} = (3 \times 3 \times 1 + 1) \times 16 = 160$$

After pool: flatten size = 3600.

Dense-1 (hidden, 128 units):

$$\text{Params}_{\text{D1}} = (3600 + 1) \times 128 = 3601 \times 128 = 460{,}928$$

Dense-2 (output, 10 units) – input is now 128:

$$\text{Params}_{\text{D2}} = (128 + 1) \times 10 = 129 \times 10 = 1{,}290$$

Total:

$$160 + 460{,}928 + 1{,}290 = \boxed{462{,}378}$$

Numerical Example – Full Worked Steps

Step	Calculation	Result
Conv params	$(9 + 1) \times 16$	**160**
Flatten size	$15 \times 15 \times 16 = 3600$	3600
Dense-1 weights	3600×128	460,800
Dense-1 biases	128	128
Dense-1 total	$460{,}800 + 128$	**460,928**
Dense-2 weights	128×10	1,280
Dense-2 biases	10	10
Dense-2 total	$1{,}280 + 10$	**1,290**
Total	$160 + 460{,}928 + 1{,}290$	**462,378**

Interpretation: Dense-1 alone contributes 99.7% of all parameters. A single large FC layer is a major bottleneck. This is why modern architectures replace FC heads with Global Average Pooling (see Architecture 7).

Code

```
model = keras.Sequential([
    keras.layers.Input(shape=(32, 32, 1)),

    # Conv: (3×3×1 + 1) × 16 = 160 params
    keras.layers.Conv2D(16, kernel_size=3, activation='relu'),

    # MaxPool: 0 params
    keras.layers.MaxPooling2D(2),

    # Flatten: 3600 values
    keras.layers.Flatten(),

    # Dense hidden: (3600 + 1) × 128 = 460,928 params
    keras.layers.Dense(128, activation='relu'),
```

```
    # Dense output: (128 + 1) × 10 = 1,290 params
    keras.layers.Dense(10, activation='softmax'),
])

model.summary()
# Total params: 462,378
```

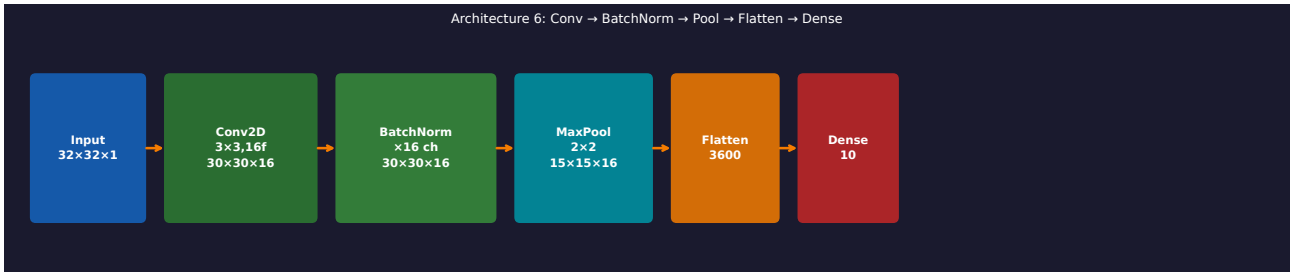
Architecture 6 – Conv → BatchNorm → Pool → Flatten → Dense

Concept

Batch Normalisation (BN) is inserted after the Conv (and before the activation in most modern designs) to stabilise training. It learns four parameters per channel: a scale (γ), a shift (β), and stores two running statistics (μ, σ^2). These add a small but non-zero parameter count.

Analogy: After each inspector writes their feature score, a quality controller (BN) re-scales and re-centres all scores so they always land in the same comfortable range – no matter which batch of images came in. The quality controller has four personal settings per inspector: preferred mean, preferred spread, and two running averages from past batches.

Visualization



Mathematical Foundation

Conv layer:

$$\text{Params}_{\text{conv}} = (3 \times 3 \times 1 + 1) \times 16 = 160$$

BatchNorm (16 channels, 4 params each):

$$\text{Params}_{\text{BN}} = 4 \times 16 = 64$$

MaxPool: 0 params.

Flatten size: 3600 (same as Architecture 2).

Dense:

$$\text{Params}_{\text{dense}} = (3600 + 1) \times 10 = 36{,}010$$

Total:

$$160 + 64 + 36{,}010 = \boxed{36{,}234}$$

Numerical Example – Full Worked Steps

Step	Calculation	Result
Conv params	$(9+1) \times 16$	**160**
BN: γ per channel	16×1	16
BN: β per channel	16×1	16
BN: μ per channel	16×1	16
BN: σ^2 per channel	16×1	16
BN total	4×16	**64**
Flatten	$15 \times 15 \times 16 = 3600$	3600
Dense params	$(3600+1) \times 10$	**36,010**
Total	$160 + 64 + 36{,}010$	**36,234**

Interpretation: BN adds only 64 parameters (0.18% overhead) but provides major training stability benefits. It is essentially free in terms of parameter count.

Code

```
model = keras.Sequential([
    keras.layers.Input(shape=(32, 32, 1)),

    # Conv: (3×3×1 + 1) × 16 = 160 params
    keras.layers.Conv2D(16, kernel_size=3, use_bias=True),

    # BatchNorm: 4 × 16 = 64 params (γ, β, μ, σ²)
    keras.layers.BatchNormalization(),
    keras.layers.Activation('relu'),

    # MaxPool: 0 params
    keras.layers.MaxPooling2D(2),

    # Flatten: 3600
    keras.layers.Flatten(),

    # Dense: (3600 + 1) × 10 = 36,010 params
    keras.layers.Dense(10, activation='softmax'),
])

model.summary()
# Total params: 36,234
```

Architecture 7 – Conv → Pool → Conv → Pool → GlobalAvgPool → Dense

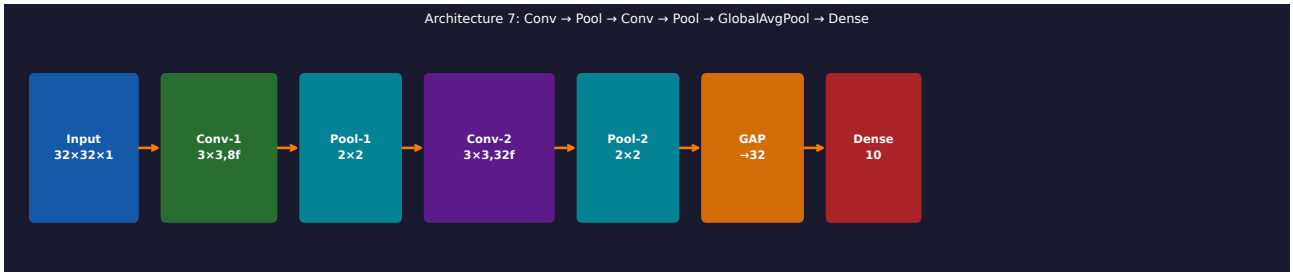
Concept

Global Average Pooling (GAP) replaces Flatten entirely. Instead of unrolling the full feature map into a vector of (H×W×C) values, GAP computes one average per channel –

producing a vector of length C, regardless of H and W. This is the dominant approach in modern networks (ResNet, MobileNet, EfficientNet).

Analogy: Instead of reading every word in every inspector's multi-page report (Flatten), GAP reads only each inspector's average score – one number per inspector. The Dense layer gets a short, clean summary of length equal to the number of filters, not the spatial size times the filters.

Visualization



Mathematical Foundation

Intuition: After two conv+pool blocks, the spatial map is 6×6×32. GAP collapses 6×6×32 → 32 (one mean per channel). Dense input is 32 – tiny compared to 576 from Flatten.

Conv-1:

$$\text{Params}_1 = (3 \times 3 \times 1 + 1) \times 8 = 80$$

Conv-2 ($C_{\text{in}} = 8$):

$$\text{Params}_2 = (3 \times 3 \times 8 + 1) \times 32 = 73 \times 32 = 2{,}336$$

GAP: 0 parameters.

Dense:

$$\text{Params}_{\text{dense}} = (32 + 1) \times 10 = 330$$

Total:

$$80 + 2{,}336 + 330 = \boxed{2{,}746}$$

Numerical Example – Full Worked Steps

Step	Calculation	Result
Conv-1	$(9 \times 1 + 1) \times 8$	**80**
After Conv-1 + Pool-1	$30 \to 15$	15×15×8
Conv-2	$(9 \times 8 + 1) \times 32 = 73 \times 32$	**2,336**
After Conv-2 + Pool-2	$13 \to 6$	6×6×32
GAP output	one mean per channel	**32 values, 0 params**
Dense	$(32 + 1) \times 10$	**330**
Total	$80 + 2{,}336 + 330$	**2,746**

Interpretation: Replacing Flatten (which gave 576 inputs to Dense) with GAP (32 inputs) shrinks the Dense layer from 5,770 to 330 params. Total dropped from 7,018 (Architecture 4) to 2,746. GAP produces a **60% reduction** in total params for this example.

Code

```
model = keras.Sequential([
    keras.layers.Input(shape=(32, 32, 1)),

    keras.layers.Conv2D(8, kernel_size=3, activation='relu'),     # 80
↪   params
    keras.layers.MaxPooling2D(2),

    keras.layers.Conv2D(32, kernel_size=3, activation='relu'),    # 2,336
↪   params
    keras.layers.MaxPooling2D(2),

    # Global Average Pooling: collapses H×W → 1 per channel → output
↪    shape (None, 32)
    # 0 params - no learnable values
    keras.layers.GlobalAveragePooling2D(),

    # Dense input size is now just 32 (not 576)
    # (32 + 1) × 10 = 330 params
    keras.layers.Dense(10, activation='softmax'),
])

model.summary()
# Total params: 2,746
```

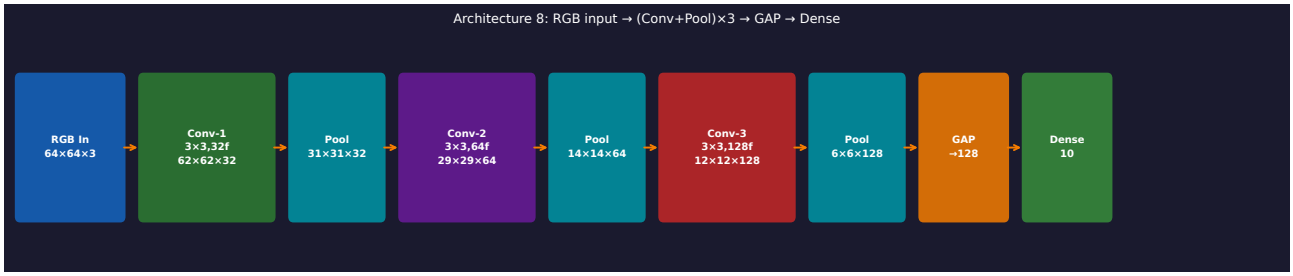
Architecture 8 – Three Conv Blocks + GAP → Dense (RGB input)

Concept

This architecture takes a **colour (RGB) image** – $C_{\text{in}} = 3$ – and passes it through three conv+pool blocks with growing filter counts (32→64→128), then uses GAP. This is a realistic backbone pattern seen in small custom CNNs.

Key point: The very first conv layer has $C_{\text{in}} = 3$ because the input has three colour channels.

Visualization



Mathematical Foundation

Conv-1 ($C_{\text{in}} = 3$, 32 filters):

$$\text{Params}_1 = (3 \times 3 \times 3 + 1) \times 32 = 28 \times 32 = 896$$

Conv-2 ($C_{\text{in}} = 32$, 64 filters):

$$\text{Params}_2 = (3 \times 3 \times 32 + 1) \times 64 = 289 \times 64 = 18{,}496$$

Conv-3 ($C_{\text{in}} = 64$, 128 filters):

$$\text{Params}_3 = (3 \times 3 \times 64 + 1) \times 128 = 577 \times 128 = 73{,}856$$

GAP: 0 params $\rightarrow$ output is 128 values.
Dense:

$$\text{Params}_{\text{dense}} = (128 + 1) \times 10 = 1{,}290$$

Total:

$$896 + 18{,}496 + 73{,}856 + 1{,}290 = \boxed{94{,}538}$$

Numerical Example – Full Worked Steps

Step	Calculation	Result
Conv-1	$(9 \times 3 + 1) \times 32 = 28 \times 32$	**896**
Conv-2	$(9 \times 32 + 1) \times 64 = 289 \times 64$	**18,496**
Conv-3	$(9 \times 64 + 1) \times 128 = 577 \times 128$	**73,856**
GAP	0	**0**
Dense	$(128 + 1) \times 10$	**1,290**
Total	$896 + 18{,}496 + 73{,}856 + 1{,}290$	**94,538**

Interpretation: Notice how Conv-3 alone (73,856) accounts for 78% of all parameters. As filters double each block and input channels grow, the conv layers become the dominant cost – not the Dense head. This is the opposite pattern from architectures without GAP.

Code

```
model = keras.Sequential([
    keras.layers.Input(shape=(64, 64, 3)),   # RGB input

    # Conv-1: C_in=3, 32 filters → (9×3+1)×32 = 896 params
    keras.layers.Conv2D(32, kernel_size=3, activation='relu'),
    keras.layers.MaxPooling2D(2),

    # Conv-2: C_in=32, 64 filters → (9×32+1)×64 = 18,496 params
    keras.layers.Conv2D(64, kernel_size=3, activation='relu'),
    keras.layers.MaxPooling2D(2),
```

```
    # Conv-3: C_in=64, 128 filters → (9×64+1)×128 = 73,856 params
    keras.layers.Conv2D(128, kernel_size=3, activation='relu'),
    keras.layers.MaxPooling2D(2),

    # GAP: collapses to 128 values regardless of spatial size
    keras.layers.GlobalAveragePooling2D(),

    # Dense: (128+1) × 10 = 1,290 params
    keras.layers.Dense(10, activation='softmax'),
])

model.summary()
# Total params: 94,538
```

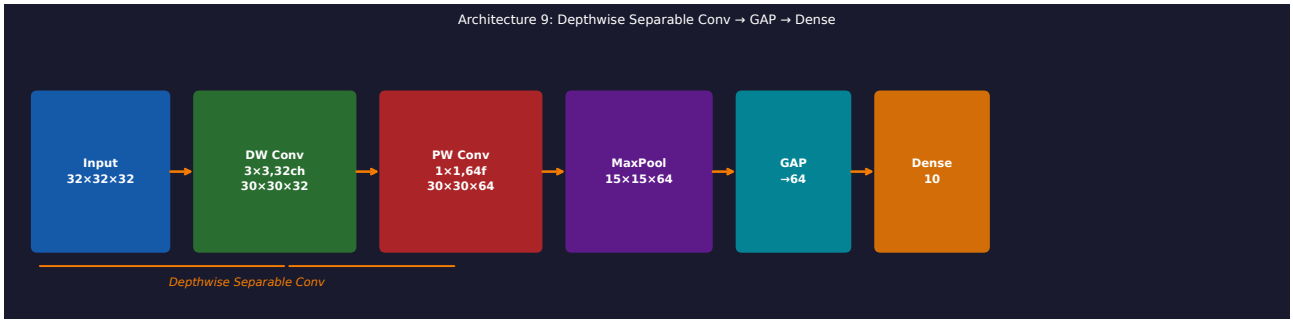
Architecture 9 – Depthwise Separable Conv → GAP → Dense

Concept

A **depthwise separable convolution** replaces a standard conv with two cheaper operations: (1) a **depthwise conv** that applies one filter per input channel independently (no cross-channel mixing), and (2) a **pointwise conv** (1×1 conv) that mixes channels. Together they approximate a full conv at a fraction of the parameter cost. This is the core building block of MobileNet.

Analogy: A standard conv is like hiring one specialist for each output report who must read every input document (cross-channel). A depthwise separable approach first assigns one reviewer per input document (depthwise – cheap, per-channel), then hires a single integrator who reads all reviewers' summaries and writes the output reports (pointwise).

Visualization



Mathematical Foundation

Standard conv equivalent for comparison ($C_{\text{in}} = 32$, $C_{\text{out}} = 64$, $K = 3$):

$$\text{Params}_{\text{standard}} = (3 \times 3 \times 32 + 1) \times 64 = 289 \times 64 = 18{,}496$$

Depthwise conv (one 3×3 filter per input channel, no mixing):

$$\text{Params}_{\text{DW}} = (3 \times 3 + 1) \times 32 = 10 \times 32 = 320$$

Pointwise conv (1×1 conv, $C_{\text{in}} = 32$, $C_{\text{out}} = 64$):

$$\text{Params}_{\text{PW}} = (1 \times 1 \times 32 + 1) \times 64 = 33 \times 64 = 2{,}112$$

GAP: 0 params → output is 64 values.

Dense:

$$\text{Params}_{\text{dense}} = (64 + 1) \times 10 = 650$$

Total:

$$320 + 2{,}112 + 650 = \boxed{3{,}082}$$

Numerical Example – Full Worked Steps

Step	Calculation	Result
DW Conv weights	$3 \times 3 \times 32 = 288$ (9 per channel, 32 channels)	288
DW Conv biases	32 (one per channel)	32
DW Conv total	$288 + 32$	**320**
PW Conv weights	$1 \times 1 \times 32 \times 64 = 2{,}048$	2,048
PW Conv biases	64	64
PW Conv total	$2{,}048 + 64$	**2,112**
Standard conv (reference)	289×64	**18,496**
Savings	$18{,}496 - 2{,}432 = 16{,}064$	**87% fewer params**
GAP	0	0
Dense	$(64 + 1) \times 10$	**650**
Total	$320 + 2{,}112 + 650$	**3,082**

Interpretation: Depthwise separable conv uses only 2,432 parameters where a standard conv would need 18,496 – an **87% reduction** while achieving similar representational power. This is why MobileNet runs efficiently on mobile devices.

Code

```
model = keras.Sequential([
    keras.layers.Input(shape=(32, 32, 32)),  # 32 input channels

    # Depthwise Separable Conv = DepthwiseConv2D + Conv2D(1×1)
    # DW: (3×3 + 1) × 32 = 320 params
    keras.layers.DepthwiseConv2D(kernel_size=3, activation='relu'),

    # PW: (1×1×32 + 1) × 64 = 2,112 params
    keras.layers.Conv2D(64, kernel_size=1, activation='relu'),

    keras.layers.MaxPooling2D(2),

    # GAP: 64 output values, 0 params
    keras.layers.GlobalAveragePooling2D(),

    # Dense: (64 + 1) × 10 = 650 params
```

```
    keras.layers.Dense(10, activation='softmax'),
])

model.summary()
# Total params: 3,082
```

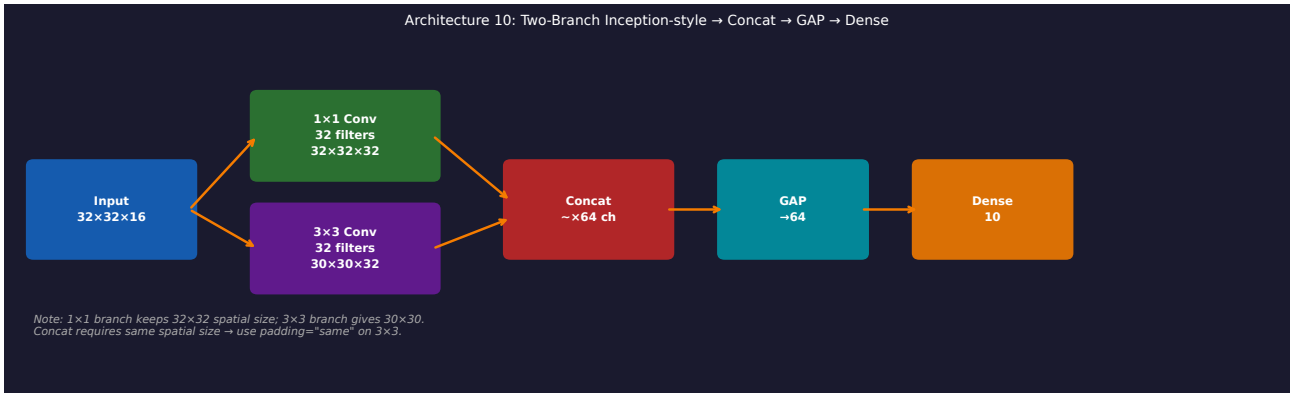
Architecture 10 – Two-Branch (Inception-style) → Concat → GAP → Dense

Concept

An **Inception-style** block runs two parallel conv branches on the same input: one with a 1×1 conv (cheap, captures point-wise patterns) and one with a 3×3 conv (captures spatial patterns). Their outputs are **concatenated** along the channel axis. Concatenation has no parameters – it just joins the feature maps. This lets the network capture features at multiple scales simultaneously.

Analogy: Two teams of inspectors work the same image in parallel – one team uses 1×1 magnifiers (local, detail-focused), another uses 3×3 magnifiers (context-aware). After both finish, their report sheets are stacked together (concat). A supervisor (GAP) then averages each inspector's stack, and the judge (Dense) makes the final call.

Visualization



Mathematical Foundation

Branch 1 – 1×1 Conv ($C_{\text{in}} = 16$, 32 filters):

$$\text{Params}_{B1} = (1 \times 1 \times 16 + 1) \times 32 = 17 \times 32 = 544$$

Branch 2 – 3×3 Conv ($C_{\text{in}} = 16$, 32 filters, `padding='same'` to match spatial size):

$$\text{Params}_{B2} = (3 \times 3 \times 16 + 1) \times 32 = 145 \times 32 = 4{,}640$$

Concat: 0 parameters. Output channels $= 32 + 32 = 64$.

GAP: 0 params → output is 64 values.

Dense:

$$\text{Params}_{\text{dense}} = (64 + 1) \times 10 = 650$$

Total:

$$544 + 4{,}640 + 650 = \boxed{5{,}834}$$

Numerical Example – Full Worked Steps

Step	Calculation	Result
B1: 1×1 Conv	$(1 \times 1 \times 16 + 1) \times 32 = 17 \times 32$	**544**
B2: 3×3 Conv	$(3 \times 3 \times 16 + 1) \times 32 = 145 \times 32$	**4,640**
Concat	$32 + 32 = 64$ channels, 0 params	**0**
GAP	0	**0**
Dense	$(64 + 1) \times 10 = 65 \times 10$	**650**
Total	$544 + 4{,}640 + 650$	**5,834**

Comparison with single 3×3 conv to same output channels (64 filters from 16 channels): $(9 \times 16 + 1) \times 64 = 145 \times 64 = 9{,}280$ params for a single branch. The two-branch design with 64 total output channels costs only 5,834 – and captures two scales of features simultaneously.

Interpretation: The 1×1 branch is very cheap (544 params) but contributes 32 feature maps capturing channel interactions at each pixel. The 3×3 branch is more expensive (4,640) but adds spatial context. Together they give the network multi-scale perception for moderate cost.

Code

```
import tensorflow as tf
from tensorflow import keras

# Functional API needed for branches
inputs = keras.Input(shape=(32, 32, 16))

# Branch 1: 1×1 conv → point-wise features
# (1×1×16 + 1) × 32 = 544 params
b1 = keras.layers.Conv2D(32, kernel_size=1, activation='relu',
 ↪  padding='same')(inputs)

# Branch 2: 3×3 conv → spatial features
# (3×3×16 + 1) × 32 = 4,640 params
# padding='same' keeps spatial size identical to Branch 1 for concat
b2 = keras.layers.Conv2D(32, kernel_size=3, activation='relu',
 ↪  padding='same')(inputs)

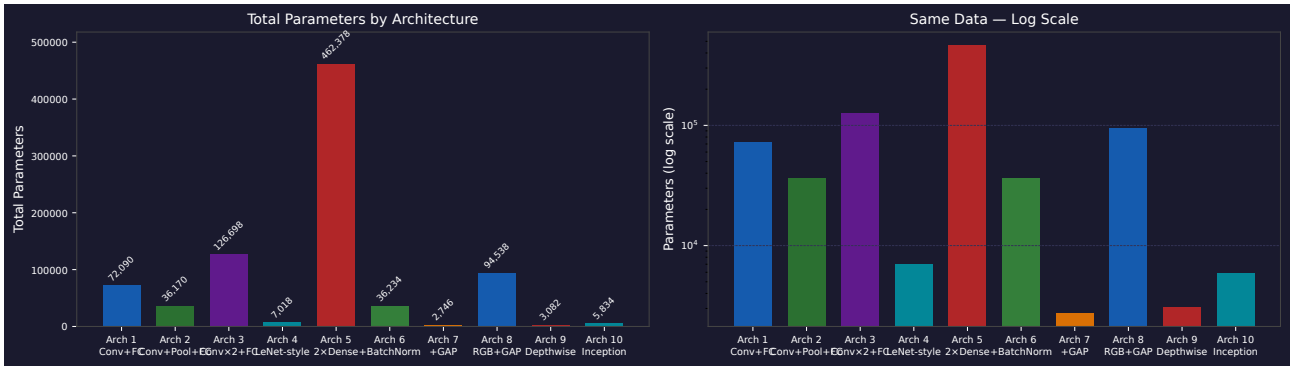
# Concat along channel axis: 32 + 32 = 64 channels, 0 params
x = keras.layers.Concatenate(axis=-1)([b1, b2])

# GAP: collapses H×W → 1 per channel → 64 values, 0 params
x = keras.layers.GlobalAveragePooling2D()(x)

# Dense: (64 + 1) × 10 = 650 params
outputs = keras.layers.Dense(10, activation='softmax')(x)
```

```
model = keras.Model(inputs, outputs)
model.summary()
# Total params: 5,834
```

Summary Comparison – All 10 Architectures



Architecture	Key Features	Total Params	Dense Head Cost
1 – Conv + Flatten + Dense	Baseline	72,090	72,010
2 – + MaxPool	Pool halves spatial	36,170	36,010
3 – Conv×2, no pool	Stacked conv	126,698	125,450
4 – LeNet-style	Conv+Pool ×2	7,018	5,770
5 – Two Dense layers	Hidden FC-128	462,378	462,218
6 – + BatchNorm	BN adds 64 params	36,234	36,010
7 – GAP replaces Flatten	GAP → tiny Dense	2,746	330
8 – RGB + 3 blocks + GAP	Conv dominates	94,538	1,290
9 – Depthwise Sep + GAP	MobileNet-style	3,082	650
10 – Two-branch + GAP	Inception-style	5,834	650

Key takeaways:

- Without pooling or GAP, the Dense head dominates (>99% of params)
- GAP is the single most powerful technique for reducing the Dense head
- Depthwise separable conv cuts conv-layer params by ~87% vs standard conv
- BatchNorm adds a negligible number of parameters
- Multi-branch (Inception) designs allow multi-scale features at moderate cost